

# Принципы ООП

Инкапсуляция, полиморфизм,  
наследование

Некрасов Ярослав Алексеевич

## 4 pillars of OOP

### Encapsulation



### polymorphism



### inheritance



### Abstraction



ProgrammerHumor.io

# Инкапсуляция



# Добавили мы новый метод

## И все могут его использовать

```
java - Person.java

1  public class Person {
2      String name;
3      int age;
4
5      public Person(String name, int age) {
6          this.name = name;
7          this.age = age;
8      }
9
10     // Новый метод
11     public void aboutMyself() {
12         System.out.println("Я люблю аниме");
13     }
14 }
```

```
java - Main.java

1  public class Main {
2      public static void main(String[] args) {
3          Person person = new Person("yarik", 21);
4          person.aboutMyself(); // Я люблю аниме
5      }
6  }
```

Нехорошо, что у всех теперь есть доступ к нашим секретным данным

# Добавили мы новый метод

## Скроем его

```
java - Person.java

1  public class Person {
2      String name;
3      int age;
4
5      public Person(String name, int age) {
6          this.name = name;
7          this.age = age;
8      }
9
10     // Поменяли public -> private
11     private void aboutMyself() {
12         System.out.println("Я люблю аниме");
13     }
14 }
```

```
java - Main.java

1  public class Main {
2      public static void main(String[] args) {
3          Person person = new Person("yarik", 21);
4          person.aboutMyself(); // Ошибка
5      }
6  }
```

Мы можем скрыть наши секреты ото всех

# Добавили мы новый метод

## Скроем его ото всех

```
java - Person.java

1  public class Person {
2      String name;
3      int age;
4
5      public Person(String name, int age) {
6          this.name = name;
7          this.age = age;
8          aboutMyself(); // <--
9      }
10
11     private void aboutMyself() {
12         System.out.println("Я люблю аниме");
13     }
14 }
```

При этом сами мы можем все использовать внутри класса

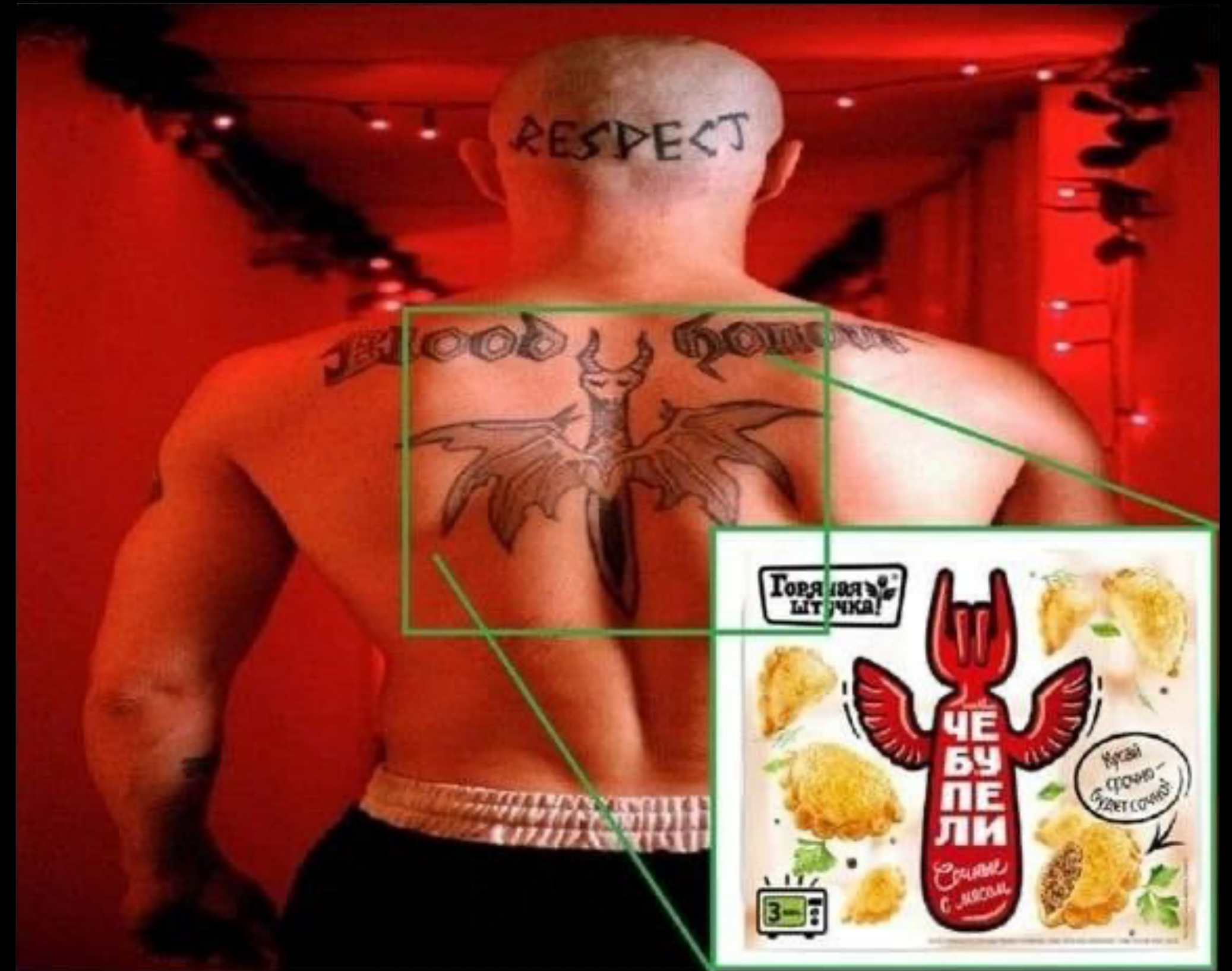
# Private

Можно скрыть не только метод, но и значение

```
java - Person.java
1  public class Person {
2      String name;
3      int age;
4      private String password; // <--
5
6      public Person(String name, int age, String password) {
7          this.name = name;
8          this.age = age;
9          this.password = password;
10     }
11 }
```

```
java - Main.java
1  public class Main {
2      public static void main(String[] args) {
3          Person person = new Person("yarik", 21, "qwerty");
4          person.password; // Ошибка
5      }
6  }
```

# Наследование



# У нас есть класс

## Как его менять?

Если мы захотим добавить новое поле в Person, то нужно будет чинить весь код, который от него зависит

```
java - Person.java

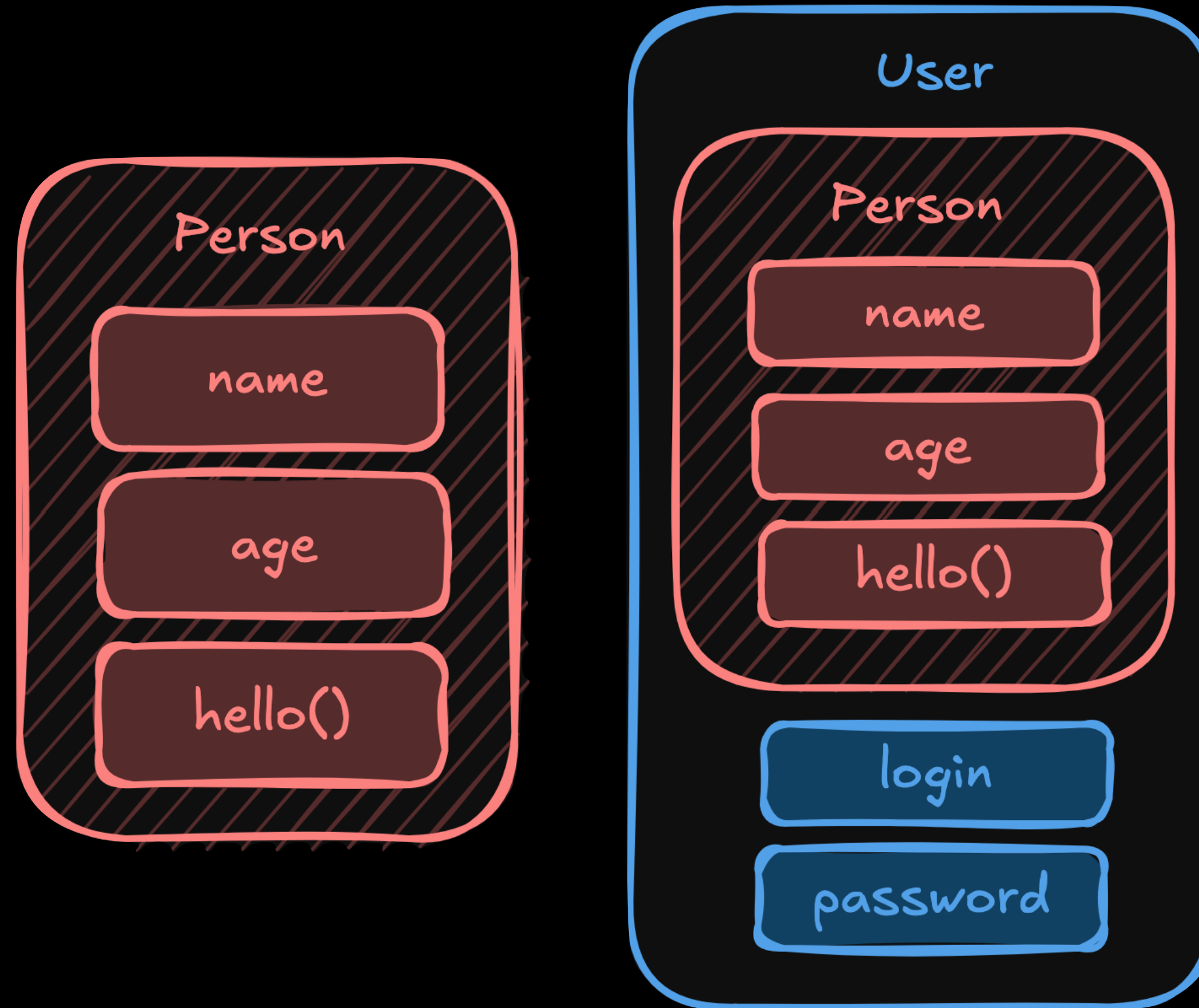
1  public class Person {
2      String name;
3      int age;
4
5      public Person(String name, int age) {
6          this.name = name;
7          this.age = age;
8      }
9
10     public void hello() {
11         System.out.println("Hello i'm " + name);
12     }
13 }
```

```
java - Main.java

1  public class Main {
2      public static void main(String[] args) {
3          Person person = new Person("yarik", 21);
4          person.hello();
5      }
6  }
```

# Наследование

В чем идея?



Нужно создать класс, который расширяет существующий, но не меняет его

# Наследование

```
java - User.java

1  public class User extends Person {
2      // Новые значения
3      String login;
4      String password;
5
6      public User(String name, int age, String login, String password) {
7          super(name, age); // <-- Создаем Person
8          this.login = login; // <-- Добавляем к Person login
9          this.password = password; // <-- Добавляем к Person password
10     }
11 }
```

Нужно использовать *super(...)* в конструкторе - это вызовет конструктор класса, который мы наследуем

# Наследование

## Используем новый класс

Старый класс никак не изменился, а новый МОЖЕТ все то же, и что-то свое



java - Main.java

```
1  public class Main {  
2      public static void main(String[] args) {  
3          Person person = new Person("yarik", 21);  
4          person.hello();  
5  
6          User user = new User("lesha", 22, "leha_", "123");  
7          user.hello();  
8      }  
9  }
```

# Наследование

## Override

```
java - User.java
1  public class User extends Person {
2      String login;
3      String password;
4
5      public User(String name, int age, String login, String password) {
6          super(name, age);
7          this.login = login;
8          this.password = password;
9      }
10
11     @Override // <-- Переписываем существующий метод
12     public void hello() {
13         System.out.println("Hi, i'm " + login);
14     }
15 }
```

Если метод из родительского класса нам не нравится - мы можем его переписать

# Наследование

## Override

Результат



java - Main.java

```
1  public class Main {
2      public static void main(String[] args) {
3          Person person = new Person("yarik", 21);
4          person.hello(); // Hello i'm yarik
5
6          User user = new User("lesha", 22, "leha_", "123");
7          user.hello(); // Hi, i'm leha_
8      }
9  }
```

# Наследование

## Override + super

```
java - User.java

1  public class User extends Person {
2      String login;
3      String password;
4
5      public User(String name, int age, String login, String password) {
6          super(name, age);
7          this.login = login;
8          this.password = password;
9      }
10
11     @Override
12     public void hello() {
13         super.hello(); // <-- вызов person.hello
14         System.out.println("my login is " + login);
15     }
16 }
```

Мы можем вызывать методы из родительского класса через super

# Наследование

## Override + super

### Использование



java - Main.java

```
1  public class Main {
2      public static void main(String[] args) {
3          User user = new User("yarik", 21, "yar1kk", "123");
4          user.hello();
5          // Hello i'm yarik
6          // my login is yar1kk
7      }
8  }
```

# Полиморфизм



# Полиморфизм

Функции ведут себя по разному в зависимости от контекста

Все функции, которые принимают родительский класс - могут принять и дочерний

```
java - Main.java

1  public class Main {
2      public static void hello(Person person) {
3          person.hello();
4      }
5
6      public static void main(String[] args) {
7          Person person = new Person("yarik", 21);
8          hello(person); // Hello i'm yarik
9
10         User user = new User("lesha", 22, "leha_", "123");
11         hello(user); // Hi, i'm leha_
12     }
13 }
```

# Полиморфизм

## Перегрузка

Мы можем создавать функции, которые называются одинаково, но различаются параметрами

Java сама выберет нужную функцию

```
java - Main.java
1  public class Main {
2      public static void hello(Person person) {
3          person.hello();
4      }
5
6      public static void hello(Person person1, Person person2) {
7          person1.hello();
8          person2.hello();
9      }
10
11     public static void main(String[] args) {
12         Person person = new Person("yarik", 21);
13         User user = new User("lesha", 22, "leha_", "123");
14
15         hello(person);
16         hello(person, user);
17     }
18 }
```

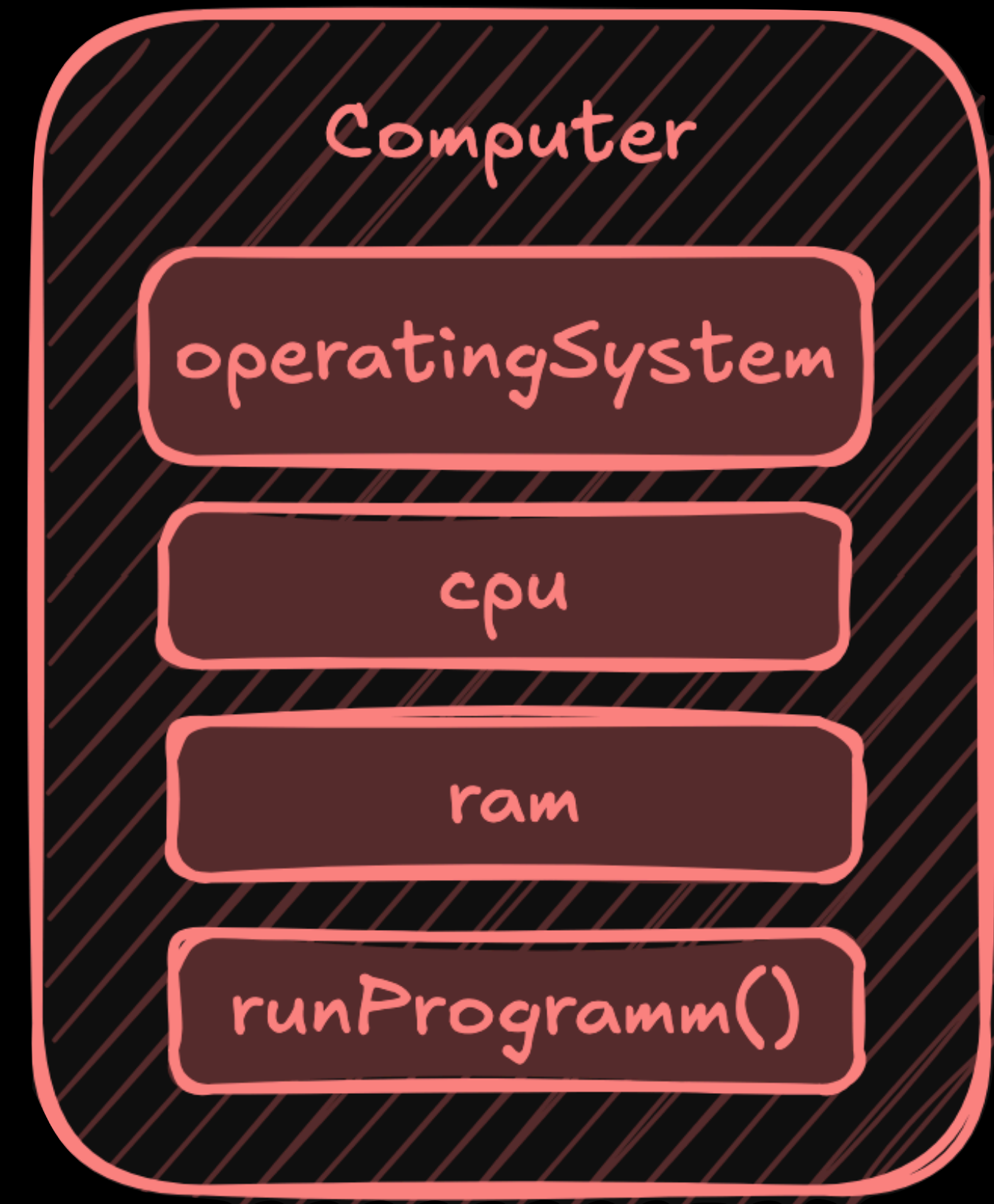
# Задание



# Задание

## За респект

Напишите класс «Computer», у которого есть поля для характеристик и метод, чтобы запускать программы

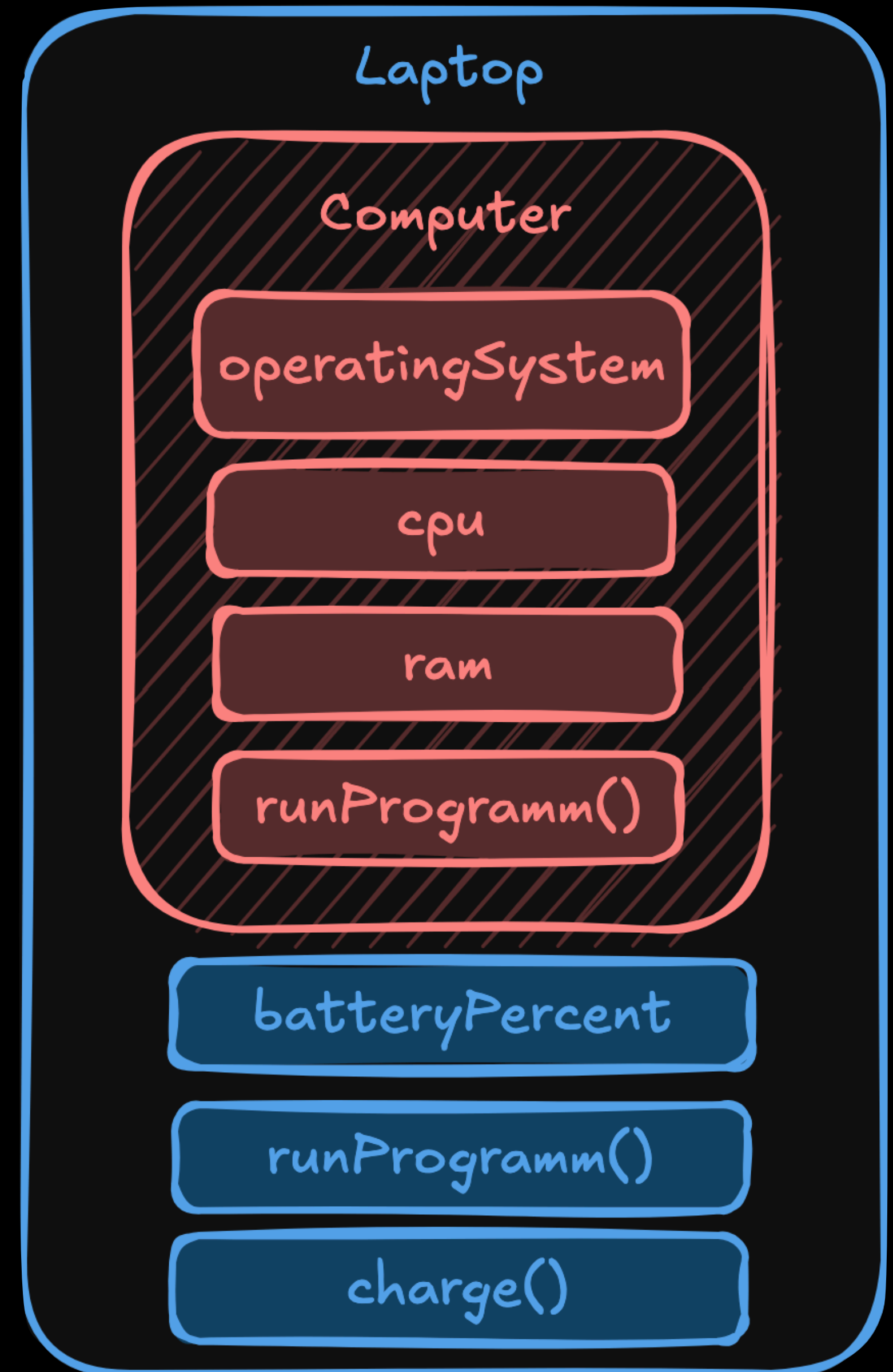


# Задание

## За респект

Класс «Laptop» должен наследовать «Computer» и:

- 1.Добавлять поле для процента заряда батареи
- 2.Метод для зарядки ноутбука
- 3.Переопределять метод для запуска программ, чтобы он уменьшал заряд батареи



# Задание

## За респект

Пример использования



java - Main.java

```
1  public class Main {
2      public static void main(String[] args) {
3          Computer computer = new Computer("intel core i7", 32);
4          computer.runProgram("vs code"); // Запуск vs code
5
6          // Изначально батарея ноутбука на 100%
7          Laptop laptop = new Laptop("intel core i5", 16);
8          System.out.println(laptop.batareyCharge); // 100
9          laptop.runProgram("pycharm"); // Запуск pycharm
10         System.out.println(laptop.batareyCharge); // 90
11         laptop.charge(); // Батарея заряжена до 100
12     }
13 }
```